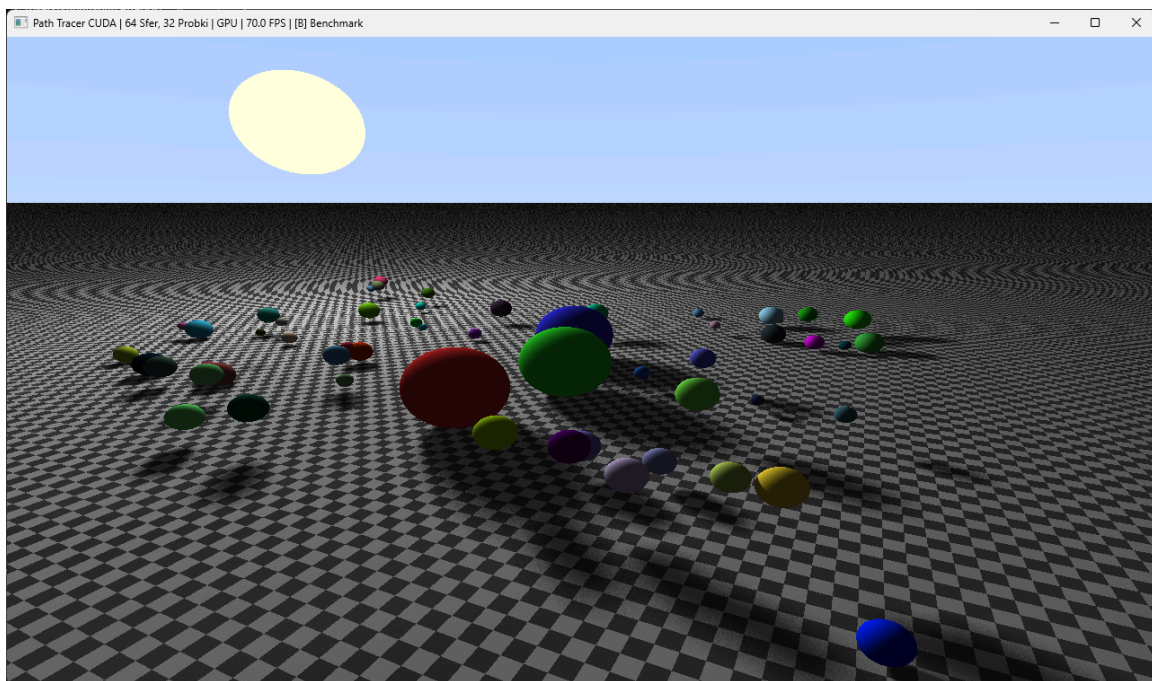


Dokumentacja Techniczna Projektu: Path Tracer CUDA Benchmark

1. Architektura Rozwiązania i Struktura Projektu

Projekt to autorski, zoptymalizowany pod kątem GPU silnik renderujący oparty na algorytmie Path Tracingu, stworzony w środowisku CUDA C++. Jego głównym zadaniem jest generowanie grafiki 3D oraz weryfikacja i porównanie wydajności obliczeniowej pomiędzy procesorem (CPU) a kartą graficzną (GPU).



Kod został podzielony na wysoce wyspecjalizowane moduły, co zapewnia czytelność i pozwala na efektywne zarządzanie pamięcią:

- **vec3.cuh:** Niskopoziomowy moduł matematyczny implementujący trójwymiarowe wektory oraz podstawowe operacje algebraiczne (dodawanie, odejmowanie, normalizacja, iloczyn skalarny i wektorowy). W pełni przystosowany do działania zarówno na hoście, jak i na urządzeniu (flagi `__host__` `__device__`).
- **physics.cuh:** Moduł fizyczny odpowiedzialny za struktury danych (Sfera, Materiał) oraz detekcję kolizji. Zawiera zoptymalizowane pod kątem GPU funkcje sprawdzające zderzenia promieni z obiektami bazujące na algebrze liniowej.
- **camera.cuh:** Moduł wirtualnej kamery z zaimplementowanym systemem nawigacji.
- **renderer.cuh:** Główny, sprzętowy rdzeń renderujący. Implementuje równoległą pętlę generowania obrazu na CPU (z użyciem OpenMP) oraz dedykowany Kernel CUDA wyznaczający ścieżki światła dla każdego piksela ekranu.

- **kernel.cu:** Kod nadrzędny pełniący funkcję sterującą. Inicjalizuje kontekst graficzny OpenGL (przy pomocy GLFW i GLEW), alokuje pamięć na karcie graficznej, wysyła dane sceny oraz zarządza zautomatyzowanym systemem benchmarkowania eksportującym dane profilujące.

2. Działanie Aplikacji, Model Fizyczny i Szczegóły Implementacji

Aplikacja opiera się na algorytmie śledzenia promieni (Path Tracing), który dla każdego piksela na ekranie symuluje fizyczne rozchodzenie się światła wstecz – od wirtualnej kamery aż do źródła światła.

2.1. Główny Cykl Renderowania i Wirtualna Kamera

Sercem silnika są funkcje render (wykonywana równolegle przez rdzenie GPU) oraz render_cpu (obsługiwana przez wątki procesora). Ich zadaniem jest mapowanie dwuwymiarowych współrzędnych przestrzeni ekranu (X, Y) na znormalizowany układ współrzędnych UV.

Kierunek wypuszczanych promieni definiuje obiekt klasy Camera (moduł camera.cuh). Funkcja process_input na bieżąco przechwytuje pozycję kursora myszy oraz wciśnięte klawisze, modyfikując pozycję kamery w przestrzeni. Następnie metoda update_vectors() przelicza kąty Eulera (Yaw i Pitch) na znormalizowane wektory kierunkowe: forward, right oraz up. Na ich podstawie, wewnątrz głównego kernela, dla każdego piksela generowany jest promień główny (struktura Ray).

2.2. Geometria Zderzeń (Ray-Sphere Intersection)

Zaraz po wygenerowaniu promienia, w pętli sprawdzane są jego zderzenia z obiektami na scenie. Fundamentem modelu fizycznego jest zaimplementowana w module physics.cuh funkcja hit_sphere_obj. Opiera się ona na matematycznym rozwiązaniu zderzenia promienia ze sferą.

Promień światła definiowany jest równaniem parametrycznym: $P(t) = A + t \cdot b$, gdzie

A to początek promienia,
b to jego znormalizowany kierunek, a
t to szukany parametr odległości.

Równanie sfery o środku C i promieniu r to: $(P - C) \cdot (P - C) = r^2$

Podstawiając równanie promienia do równania sfery i rozwijając je, algorytm wewnątrz hit_sphere_obj układa równanie kwadratowe względem odległości t. Następnie funkcja wylicza wyróżnik (Delta). Konstrukcja warunkowa sprawdza jedynie przypadki, dla których $\Delta > 0$. W takiej sytuacji promień przebija sferę – funkcja rozwiązuje równanie i zwraca najmniejszą wartość t (najbliższy punkt zderzenia). Równolegle, za pomocą funkcji hit_floor, sprawdzane jest zderzenie z nieskończoną płaszczyzną podłogi.

2.3. Model Oświetlenia i Cienie Monte Carlo

Jeśli promień uderzy w jakikolwiek obiekt (`hit_index != -1`), obliczany jest wektor normalny powierzchni w punkcie zderzenia (odpowiada za to funkcja `normalize()` z modułu `vec3.cuh`). Następnie aplikacja wylicza natężenie światła na podstawie iloczynu skalarnego (funkcja `dot()`) wektora normalnego i wektora skierowanego do źródła światła.

Aby uzyskać realistyczny efekt miękkich cieni, program wykorzystuje metodę próbkowania Monte Carlo. Uruchamiana jest wewnętrzna pętla wypuszczająca promienie wtórne z punktu zderzenia w stronę oświetlenia (ilość zdefiniowana parametrem **shadow_samples**).

Za losowe "rozproszenie" tych promieni odpowiada funkcja `rand_float()` – sprzętowy generator liczb pseudolosowych wykorzystujący operacje bitowe (XOR i przesunięcia). Dzięki temu badany jest obszar o promieniu `light_radius`, co powoduje płynne rozmycie cieni na krawędziach.

3. Przeprowadzenie Eksperymentu Wydajnościowego (Benchmark)

Częścią projektu jest wbudowany dedykowany moduł profilowania oprogramowania, uruchamiany asynchronicznie za pomocą interfejsu klawiaturowego.

1. **Testowany Scenariusz:** Skalowalna przestrzeń losowo generowanych sfer (od 1 do 64 i więcej) połączona ze zmianą dokładności miękkich cieni (od 8 do 32 próbek na każde zderzenie promienia). Geometria sfer jest ponadto dynamicznie animowana (lewitacja) przy użyciu funkcji trygonometrycznych (Dla sfer > 3).
2. **Techniki Pomiaru:** Program korzysta z niezależnych, asynchronicznych cykli profilujących. Dla karty graficznej (GPU) aplikacja wymusza generację 100 klatek z rzędu w celu eliminacji błędu statystycznego narzutu systemowego oraz stabilizacji bufora zegara. Wynik jest uśredniany.
3. **Synchronizacja Krytyczna:** Kod używa bariery `cudaDeviceSynchronize()`. Chroni to system zliczający czas przez fałszywym raportowaniem końca procesu. Zrównuje to czas działania CPU z asynchroniczną pracą GPU, czekając na twarde zakończenie renderingu na jednostkach procesora obrazu.

4. Wyniki i Porównanie Architektoniczne

System automatycznie testował zjawiska optymalizacyjne zrzucając wyniki do formatu pliku tekstowego rozdzielanego przecinkami (.csv). Zaobserwowano potężny skok wydajności wielordzeniowego przetwarzania GPU w miarę geometrycznego wzrostu skomplikowania sceny, ukazując barierę przepustowości dla tradycyjnych jednostek zmiennoprzecinkowych FPU głównego procesora.

Skrócone wyniki profilowania

Tabela 1. Wyniki dla rozdzielczości HD (1280x720)

Rozdzielczość	Sfery	Próbki	Czas GPU [ms]	FPS GPU	Czas CPU [ms]	FPS CPU	Speedup
1280x720	1	8	0.36	2803.63	23.33	42.86	65.42x
1280x720	1	16	0.49	2027.76	40.92	24.44	82.99x
1280x720	1	32	0.76	1324.08	75.76	13.20	100.32x
1280x720	3	8	0.55	1802.80	32.69	30.59	58.94x
1280x720	3	16	0.62	1621.69	57.42	17.42	93.12x
1280x720	3	32	1.12	890.26	108.72	9.20	96.79x
1280x720	16	8	1.41	710.07	237.63	4.21	168.73x
1280x720	16	16	1.71	584.99	427.88	2.34	250.30x
1280x720	16	32	3.14	318.76	786.84	1.27	250.81x
1280x720	32	8	1.75	570.91	407.59	2.45	232.69x
1280x720	32	16	3.06	326.29	907.50	1.10	296.11x
1280x720	32	32	5.04	198.27	1551.67	0.64	307.65x
1280x720	64	8	2.53	395.82	822.62	1.22	325.61x
1280x720	64	16	4.68	213.88	1499.74	0.67	320.76x
1280x720	64	32	9.85	101.55	3166.87	0.32	321.58x

Przyspieszenie GPU względem CPU
Rozdzielczość HD (1280x720) z podziałem na Jakość Cieni

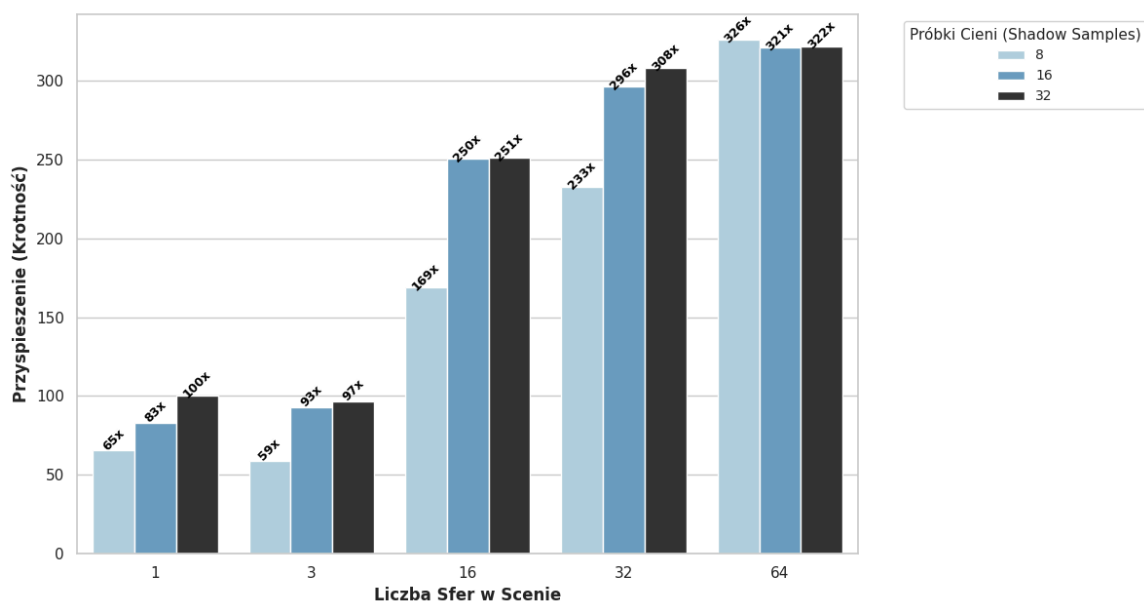
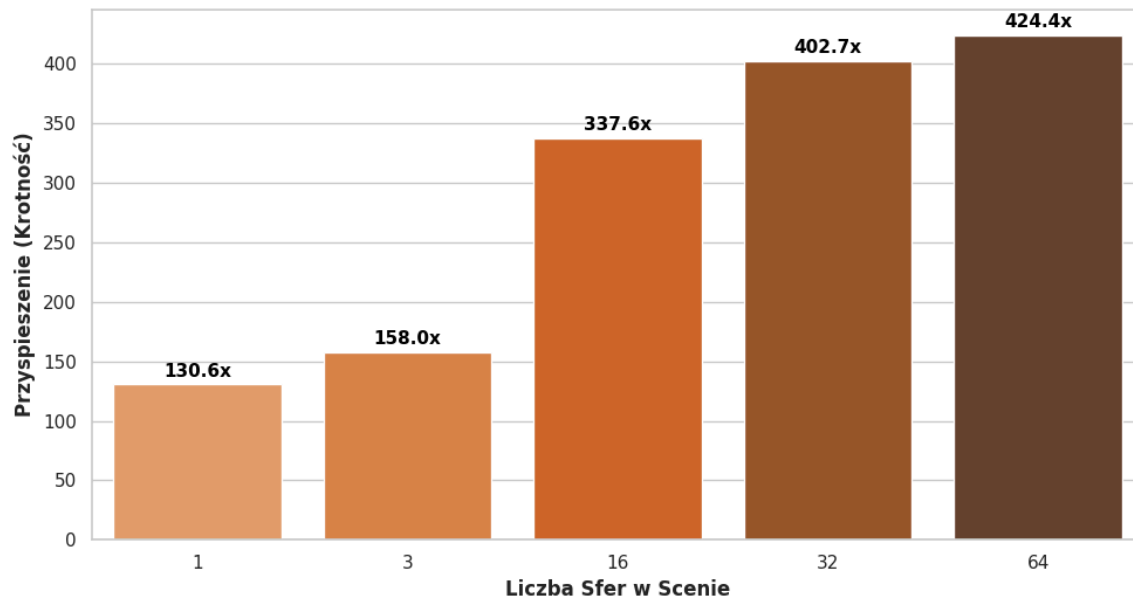


Tabela 2. Wyniki dla rozdzielczości FHD (1920x1080)

Rozdzielczość	Sfery	Próbki	Czas GPU [ms]	FPS GPU	Czas CPU [ms]	FPS CPU	Speedup
1920x1080	1	16	0.71	1408.52	92.74	10.78	130.63x
1920x1080	3	16	0.79	1268.57	124.57	8.03	158.03x
1920x1080	16	16	2.37	421.37	801.10	1.25	337.56x
1920x1080	32	16	4.04	247.79	1625.05	0.62	402.67x
1920x1080	64	16	7.76	128.85	3293.61	0.30	424.40x

**Przyspieszenie GPU względem CPU
Rozdzielczość Full HD (1920x1080) | Próbki Cieni = 16**



5. Podsumowanie

Implementacja Path Tracingu doskonale sprawdziła się jako zaawansowany problem testowy dla środowiska CUDA. Zastosowanie fundamentalnych technik Programowania CUDA obniżyło czas renderowania zaawansowanej klatki z poziomów sięgających sekund na procesorze głównym, do niskich wartości w milisekundach dla procesora GPU. Architektura CUDA dowiodła swojej nadrzędności w symulacyjnym rozpraszaniu i modelowaniu zjawisk fizycznych opartych na dużych, sparametryzowanych obciążeniach z powtarzalną logiką matematyczną.